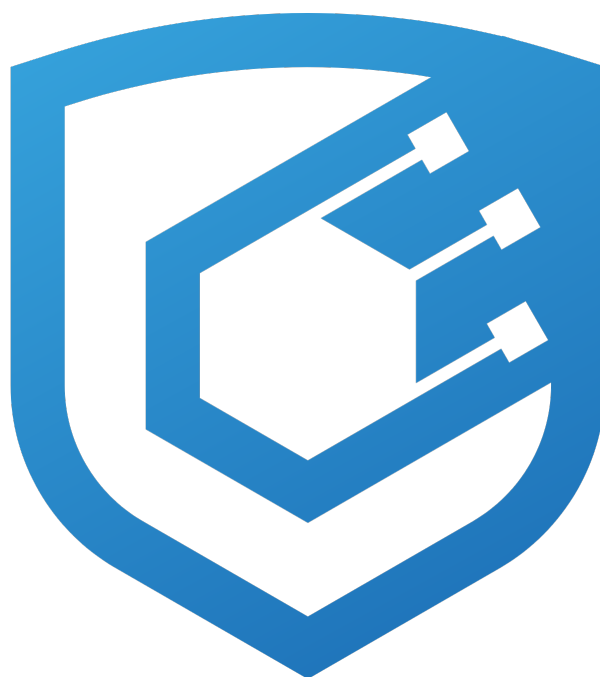




Securitize EVM GlobalDenyList Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[SBSecurity](#) ([Blckhv](#), [Slavcheww](#))

September 11, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	Compliance-path coverage completeness	3
6.2	Regression verdict against the audited baseline	3
6.3	Role and upgrade-authority assessment	3
6.4	Upgrade authority and storage layout	3
6.5	Fail-open / fail-closed review	3
6.6	Pause semantics asymmetry	3
6.7	Interface conformance across the repository boundary	3
7	Findings	6
7.1	Low Risk	6
7.1.1	GlobalDenyListManager::changeAdmin zero-admin guard is bypassed by inherited AccessControlUpgradeable::revokeRole, renounceRole, permanently bricking admin-gated functions	6
7.1.2	ComplianceServicePermissionless::checkTransfer never screens the transferFrom spender, so denylisted address can direct transfers of other wallets' tokens breaching OFAC FAQ400 compliance	7
7.1.3	Global denylist can be wired into compliance types that never check it	8
7.1.4	Sanctioned addresses can be registered as investor or special wallets in breach of OFAC FAQ42	9
7.2	Informational	11
7.2.1	Deploy task implicit GlobalDenyListManager::initialize resolution can silently deploy an uninitialized proxy	11
7.2.2	deploy-all does not await GlobalDenyListManager::addOperator, changeAdmin receipts and reports success on unconfirmed transactions	11
7.2.3	Denylist mutators always return true, discarding the EnumerableSet result	12
7.2.4	GlobalDenyListManager::revokeOperator is bypassable via inherited AccessControlUpgradeable::revokeRole, renounceRole, which emit no OperatorRevoked	13
7.2.5	ServiceConsumer::setDSService accepts an address with no code, halting all transfers and issuance on the wired token	13
7.2.6	deploy-all treats a zero --global-denylist-manager-address as wired, logging success while leaving global denylist enforcement off	14
7.2.7	ComplianceService::validateSeize screens the seize destination only for special-wallet status, so a globally denylisted special wallet can still receive seized tokens	14
7.3	Gas Optimization	16
7.3.1	Cache identical storage reads in ComplianceServicePermissionless::_isGloballyDenylisted, _isLocallyBlacklisted	16

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

DSToken is an upgradeable, rebasing ERC-20 security-token system that coordinates investor registration, transfer restrictions, compliance rules, privileged roles and issuance through a registry of supporting services. This engagement the new global deny list component and its wiring into the permissionless variant of token DSToken compliance.

5 Audit Scope

The audit scope was limited to the new [global deny list](#) repo and changes to existing DSToken for [PR102](#):

```
// new contracts
bc-global-denylist-manager-sc/contracts/denylist/GlobalDenyListManager.sol
bc-global-denylist-manager-sc/contracts/denylist/IGlobalDenyListManager.sol
bc-global-denylist-manager-sc/contracts/utils/BaseRBACContract.sol
// deployment
bc-global-denylist-manager-sc/tasks/deploy-all.ts
bc-global-denylist-manager-sc/tasks/deploy-denylist-manager.ts
bc-global-denylist-manager-sc/tasks/verify-denylist-manager.ts

// minor changes to existing contracts
dstoken/contracts/compliance/ComplianceServicePermissionless.sol
dstoken/contracts/compliance/IDSGlobalDenyListManager.sol
dstoken/contracts/service/IDSServiceConsumer.sol
// minor changes to existing deployment
dstoken/tasks/deploy-all.ts
dstoken/tasks/set-services.ts
dstoken/tasks/verify-all.ts
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the [EVM GlobalDenyList](#) code provided by [Securitize](#). The findings consist of 4 Low severity issues with the remainder being informational and gas optimizations.

6.1 Compliance-path coverage completeness

The primary compliance gap identified was fixed in finding L-2 where a sanctioned spender could direct the transfer of other wallets' tokens (if they had given approval) contrary to OFAC FAQ400 compliance requirements. For additional hardening as part of the L-2 fix, a sanctioned spender can no longer receive have their allowance increased but can still have their allowance decreased or removed.

6.2 Regression verdict against the audited baseline

The new changes are additive - they add additional restrictions in the form of the global deny list and calls to it. Hence there is no weakening of any existing code or requirements.

6.3 Role and upgrade-authority assessment

The main issue identified and fixed related to the deployment plan and roles was L-2 which could theoretically (but in practice very unlikely) result in the role handover transactions during deployment silently failing and the deployer not being aware of this.

A secondary issue L-1 was identified which contradicted the requirement "Verify no remaining sequence leaves the contract with zero admins" as it would allow a state where no admin exists, however this was acknowledged since it requires an admin to take a deliberate & avoidable action by explicitly calling an inherited function which was not overridden in the `GlobalDenyListManager` contract.

6.4 Upgrade authority and storage layout

There were no issues here and the default admin ownership can be migrated over to the timelock (under development) as part of a separate stream of work in the future.

6.5 Fail-open / fail-closed review

`GlobalDenyListManager::isGloballyDenylisted` just returns a set membership and any transfers/issuances/approvals etc should be reverted if that set membership is true, so there are no issues here. If no global deny list is configured then the current behavior continues so existing tokens are not impacted.

6.6 Pause semantics asymmetry

It seems correct that the read-only function `GlobalDenyListManager::isGloballyDenylisted` is not gated by pausing. `DSToken` has its own pausing mechanism which stops transfers via `ComplianceServicePermissionless::newPreTransferCheck` in the permissionless compliance variant intended to be used with the global deny list.

6.7 Interface conformance across the repository boundary

There are no issues or significant risks here, the interfaces just need to be kept in sync if the global deny list is updated in the future.

Summary

Project Name	EVM GlobalDenyList
Repository	bc-global-denylist-manager-sc
Commit	d96f85cfad7a...
Fix Commit	4f08c18d3f00...
Repository 2	dstoken
Commit	48da654d2447...
Fix Commit	a6852abdeb1f...
Audit Timeline	Sep 3rd - Sep 4th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	4
Informational	7
Gas Optimizations	1
Total Issues	12

Summary of Findings

[L-1] GlobalDenyListManager::changeAdmin zero-admin guard is bypassed by inherited AccessControlUpgradeable::revokeRole, renounceRole, permanently bricking admin-gated functions	Acknowledged
[L-2] ComplianceServicePermissionless::checkTransfer never screens the transferFrom spender, so denylisted address can direct transfers of other wallets' tokens breaching OFAC FAQ400 compliance	Resolved
[L-3] Global denylist can be wired into compliance types that never check it	Resolved
[L-4] Sanctioned addresses can be registered as investor or special wallets in breach of OFAC FAQ42	Acknowledged
[I-1] Deploy task implicit GlobalDenyListManager::initialize resolution can silently deploy an uninitialized proxy	Resolved
[I-2] deploy-all does not await GlobalDenyListManager::addOperator, changeAdmin receipts and reports success on unconfirmed transactions	Resolved
[I-3] Denylist mutators always return true, discarding the EnumerableSet result	Resolved

[I-4] GlobalDenyListManager::revokeOperator is bypassable via inherited AccessControlUpgradeable::revokeRole, renounceRole, which emit no OperatorRevoked	Resolved
[I-5] ServiceConsumer::setDSService accepts an address with no code, halting all transfers and issuance on the wired token	Partially Resolved
[I-6] deploy-all treats a zero -global-denylist-manager-address as wired, logging success while leaving global denylist enforcement off	Resolved
[I-7] ComplianceService::validateSeize screens the seize destination only for special-wallet status, so a globally denylisted special wallet can still receive seized tokens	Acknowledged
[G-1] Cache identical storage reads in ComplianceServicePermissionless::_isGloballyDenylisted, _isLocallyBlacklisted	Resolved

7 Findings

7.1 Low Risk

7.1.1 GlobalDenylistManager::changeAdmin **zero-admin guard is bypassed by inherited AccessControlUpgradeable::revokeRole, renounceRole, permanently bricking admin-gated functions**

Description: GlobalDenylistManager::changeAdmin enforces that a valid admin always exists: addressNotZero rejects address(0) and CannotTransferAdminToSelf rejects a self-target, with an in-code comment spelling out that zero admins would brick every gated function. GlobalDenylistManager::grantRole is likewise overridden to add onlyRole and addressNotZero.

revokeRole and renounceRole are inherited from AccessControlUpgradeable and are not overridden. DEFAULT_ADMIN_ROLE is its own role admin, so the sole admin can strip their own role through either path: renounceRole only requires callerConfirmation to equal msg.sender, and revokeRole only requires the caller to hold getRoleAdmin(DEFAULT_ADMIN_ROLE), which the admin satisfies by definition. The invariant is enforced on one of the three paths that can remove it.

Impact: After either call, GlobalDenylistManager::isAdmin returns false for the former admin and no account holds DEFAULT_ADMIN_ROLE. Every admin-gated function becomes permanently unreachable: changeAdmin, addOperator, revokeOperator, grantRole, and BaseRBACContract::pause, unpause, _authorizeUpgrade. No admin can be reinstated, and because _authorizeUpgrade is itself admin-gated, no upgrade can be authorized to recover.

The worst case combines this with the pause state. An admin that pauses and then renounces leaves the contract permanently paused: operators cannot call addToGlobalDenylist or removeFromGlobalDenylist because of whenNotPaused, no admin exists to unpause, and no admin exists to authorize an upgrade out of that state. The global denylist is frozen with no recovery path.

This breaches section 4.2 Role separation is procedural, not enforced from GlobalDenylist-AuditScope.md which says:

- changeAdmin guards self-transfer (CannotTransferAdminToSelf) and performs grant and revoke as independent, non-short-circuited statements. Verify no remaining sequence leaves the contract with zero admins, or silently retains the deployer as admin.

Recommended Mitigation: Route every DEFAULT_ADMIN_ROLE transition through changeAdmin by rejecting that role in both inherited entry points:

```
function revokeRole(bytes32 role, address account) public virtual override onlyRole(DEFAULT_ADMIN_ROLE)
{
    if (role == DEFAULT_ADMIN_ROLE) revert CannotRevokeAdminRole();
    super.revokeRole(role, account);
}

function renounceRole(bytes32 role, address callerConfirmation) public virtual override {
    if (role == DEFAULT_ADMIN_ROLE) revert CannotRenounceAdminRole();
    super.renounceRole(role, callerConfirmation);
}
```

Also consider whether more than one admin should exist; currently grantRole accepts an arbitrary role so grantRole(DEFAULT_ADMIN_ROLE, account) can already create a second admin.

If more than one admin is intended the above recommended mitigation should instead block removal of the last remaining admin, which AccessControlEnumerableUpgradeable supports through getRoleMemberCount.

Securitize: Acknowledged; this scenario requires the admin to take a deliberate, avoidable action against their own interest. changeAdmin (the intended, guarded path) carries an equivalent self-inflicted risk that no code change eliminates: transferring admin to the wrong address also permanently locks the caller out, with the same blast radius (no admin, no path to _authorizeUpgrade). Restricting revokeRole/renounceRole narrows one specific way to make that mistake without addressing the underlying class of risk.

There's also a real cost on the other side: this contract can technically hold more than one `DEFAULT_ADMIN_ROLE` account (nothing prevents `grantRole(DEFAULT_ADMIN_ROLE, account)` from creating a second admin). If that ever happens, blocking `revokeRole/renounceRole` removes the only way to remove a specific admin without their cooperation; `changeAdmin` only ever transfers the caller's own role, it cannot target-remove another admin. So the mitigation trades a self-inflicted single-admin mistake (already possible via `changeAdmin`) for a real operational dead-end in the multi-admin case.

Given both paths carry comparable operator-error risk, and the fix actively removes a needed capability in the multi-admin case, we're leaving `revokeRole/renounceRole` as inherited from `AccessControlUpgradeable`, unguarded for `DEFAULT_ADMIN_ROLE`.

7.1.2 `ComplianceServicePermissionless::checkTransfer` never screens the `transferFrom` spender, so denylisted address can direct transfers of other wallets' tokens breaching OFAC FAQ400 compliance

Description: `DSToken::transferFrom` applies the `canTransfer` modifier, which forwards only the token owner and the recipient into `validateTransfer`. `msg.sender`, the spender exercising the allowance, never reaches the compliance layer, so `ComplianceServicePermissionless::checkTransfer` evaluates the denylist against `_from` and `_to` only.

A globally denylisted address holding an allowance from a clean owner can therefore call `transferFrom` and move that owner's tokens to a clean recipient. `DSToken::approve` carries no compliance check either, so an allowance can also be granted to an address after it has been denylisted.

Impact: No value flows to or from the denylisted address, so this is not an asset-recovery bypass. Rather is in breach of OFAC compliance; [OFAC FAQ 400](#) explicitly forbids sanctioned entities from directing transactions:

Can persons engage in negotiations, enter into contracts, or process transactions involving a blocked individual when that blocked individual is acting on behalf of the non-blocked entity that he or she controls...

No. OFAC sanctions generally prohibit transactions involving, directly or indirectly, a blocked person, absent authorization from OFAC, even if the blocked person is acting on behalf of a non-blocked entity.

[OFAC's virtual-currency guidance](#) confirms the obligations apply identically on-chain, and FAQ 560 states compliance obligations are the same regardless of whether a transaction is denominated in digital or fiat currency.

A wallet-level control screening only the two ends of the value flow cannot express the FAQ 400 prohibition. If the shared denylist is intended to carry sanctions listings, an OFAC sanctioned address can still act as an authorized agent over other holders' tokens on every wired token.

The likelier subject is a contract rather than an individual: denylisting a sanctioned protocol or venue address stops it holding or receiving tokens, but not pulling tokens from every wallet that already approved it.

Recommended Mitigation: * Screen the spender on the paths where a spender exists, not inside `checkTransfer`. `checkTransfer` is reached by `preTransferCheck`, an off-chain view whose `msg.sender` is the arbitrary caller of the view and carries no meaning there:

```
function transferFrom(address _from, address _to, uint256 _value)
    public virtual override canTransfer(_from, _to, _value) returns (bool)
{
+   require(!getComplianceService().isGloballyDenylistedWallet(msg.sender), "Spender is globally
→   denylisted");
    return postTransferImpl(super.transferFrom(_from, _to, _value), _from, _to, _value);
}
```

- the above check automatically applies to `transferWithPermit`, but also consider whether `approve` and the allowance-increase functions should also reject a denylisted spender, so fresh authority cannot be granted to an OFAC designated address after listing
- Have `GlobalDenyListManager::isGloballyDenylisted` additionally call `0x40C57923924B5c5c5455c48D93317139ADD` to easily reject transfers involving OFAC sanctioned entities without needing to maintain a duplicated list

- Consider adding a function `GlobalDenyListManager::isGloballyDenylisted(address[] calldata wallets)` such that callers can make only one external call to check a list of input addresses. For example when checking a token transfer, one external call could be made to check `spender`, `from`, `to` which is more efficient than making 3 external calls

If it is by design that spender bypasses the deny list breaching OFAC compliance, this should be explicitly documented since the omission is otherwise indistinguishable from an oversight.

Securitize: Fixed in commits [4e0b2dd](#), [a6852ab](#) and added bulk deny list check in commit [a96f979](#).

Cyfrin: Verified.

7.1.3 Global denylist can be wired into compliance types that never check it

Description: The `deploy-all` task takes `--compliance` and `--global-denylist-manager-address` as independent inputs. `getComplianceContractName` picks the compliance contract from the first one.

<https://github.com/securitize-io/bc-global-denylist-manager-sc/blob/main/dstoken/tasks/utils/task.helper.ts#L19-L32>

```
export const getComplianceContractName = (complianceType: string): string => {
  switch (complianceType) {
    case 'WHITELISTED':
      return 'ComplianceServiceWhitelisted';
    case 'GLOBAL_WHITELISTED':
      return 'ComplianceServiceGlobalWhitelisted';
    case 'BLACKLISTED':
    case 'PERMISSIONLESS':
      return 'ComplianceServicePermissionless';
    case 'REGULATED MOCK':
      return 'ComplianceServiceRegulatedMock';
    default:
      return 'ComplianceServiceRegulated';
  }
}
```

`set-services` wires the denylist address into `dsToken` and `complianceService` whenever it is given, without looking at the compliance type.

<https://github.com/securitize-io/bc-global-denylist-manager-sc/blob/main/dstoken/tasks/set-services.ts#L115-L119>

```
if (globalDenylistManager) {
  console.log('Connecting compliance service to global denylist manager');
  tx = await complianceService.setDSService(DSConstants.services.GLOBAL_DENYLIST_MANAGER,
    ↪ globalDenylistManager.getAddress());
  await tx.wait();
}
```

Only `ComplianceServicePermissionless` calls `isGloballyDenylisted`. `ComplianceServiceWhitelisted`, `ComplianceServiceGlobalWhitelisted` and `ComplianceServiceRegulated` do not reference the `GLOBAL_DENYLIST_MANAGER` slot at all. The runbook only shows the flag with `--compliance PERMISSIONLESS`, but nothing enforces that.

Impact: A token deployed with any other compliance type and a denylist address reports a successful wiring, but the denylist is never consulted. A wallet on the global denylist can still be issued tokens and transfer them.

Recommended Mitigation: Fail the deploy task when the denylist address is supplied with a compliance type that does not enforce it.

<https://github.com/securitize-io/bc-global-denylist-manager-sc/blob/main/dstoken/tasks/deploy-all.ts#L51-L57>

```

const denylistCompliance = ['PERMISSIONLESS', 'BLACKLISTED'];
if (args.globalDenylistManagerAddress && !denylistCompliance.includes(args.compliance)) {
  throw new Error(
    `--global-denylist-manager-address is only supported with --compliance PERMISSIONLESS or
    ↪ BLACKLISTED, got ${args.compliance}`
  );
}

```

Securitize: Fixed in commit [db15596](#).

Cyfrin: Verified.

7.1.4 Sanctioned addresses can be registered as investor or special wallets in breach of OFAC FAQ42

Description: A wallet address enters the protocol through one of two independent registries, and neither consults the global denylist at any point. A globally denylisted address can therefore be registered as an investor wallet or assigned a special wallet type after it has been designated.

Investor wallets are held by RegistryService. All three entry points are `onlyExchangeOrAbove`, except `WalletRegistrar::registerWallet` which is `onlyOwnerOrIssuerOrAbove`:

1. `RegistryService::addWallet` - binds one address to an existing investor, and is the single internal choke point every investor wallet passes through
2. `RegistryService::updateInvestor` - iterates its `_wallets` argument and calls `addWallet` for each address not already registered
3. `WalletRegistrar::registerWallet` - delegates directly to `updateInvestor`

Special wallets are held by `WalletManager`. All five entry points are `onlyIssuerOrAbove` and all funnel into the internal `WalletManager::setSpecialWallet`:

4. `WalletManager::addIssuerWallet` - assigns type `ISSUER`
5. `WalletManager::addIssuerWallets` - bulk form, capped at 30 addresses
6. `WalletManager::addPlatformWallet` - assigns type `PLATFORM`
7. `WalletManager::addPlatformWallets` - bulk form, capped at 30 addresses
8. `WalletManager::addExchangeWallet` - assigns type `EXCHANGE`, and additionally requires the supplied owner to hold the exchange role

Both choke points already perform registration-time validation, so the pattern of rejecting an ineligible address at the point of registration is established. `RegistryService::addWallet` rejects an address that already carries a special wallet type and rejects one holding a non-zero balance. `WalletManager::setSpecialWallet` rejects an address that already belongs to an investor and rejects a direct type change. The global denylist is simply absent from both sets of checks.

The global denylist is intended to be a protocol-wide ban expressing sanctions designations. [OFAC FAQ 42](#) addresses this situation directly and treats onboarding, rather than transacting, as the prohibited act:

What do I do if a person tries to open an account and the individual or entity's name is on OFAC's SDN List (or is otherwise a blocked person)? Do I open the account and then block the funds?

A U.S. financial institution, its foreign branches, and - in some cases - its wholly-owned or -controlled foreign subsidiaries, cannot open an account for a person named on OFAC's List of Specially Designated Nationals and Blocked Persons (SDN List) or a person who is otherwise blocked (e.g., a blocked government or an entity that is subject to the 50 Percent Rule). This is a prohibited service.

[OFAC FAQ 560](#) confirms the obligation is identical on-chain:

Are my OFAC compliance obligations the same, regardless of whether a transaction is denominated in digital currency or traditional fiat currency?

Yes, the obligations are the same.

[OFAC FAQ 1021](#) extends it explicitly to wallet-level service providers:

U.S. persons, including virtual currency exchanges, virtual wallet hosts, and other service providers ... are generally prohibited from engaging in or facilitating prohibited transactions, including virtual currency transactions in which blocked persons have an interest.

Registering a wallet address is the on-chain analogue of opening an account: it is the step that binds an address to an identity or a role inside the platform and makes it eligible to hold and move a token. The point of FAQ 42 is that the prohibition attaches at that moment, not only when value later moves. A control that screens transfers but not registration therefore permits the one step FAQ 42 names as prohibited in its own right, and today the only signal that anything is wrong appears later, when a transfer or issuance involving that wallet is rejected.

Recommended Mitigation: Before registering any new address in the protocol, consult the global deny list and revert if that address is sanctioned.

Securitize: Acknowledged; on investor wallets (`RegistryService::addWallet`) PERMISSIONLESS-compliance token (the only compliance type with a working global denylist) deploys with `StubRegistryService` by default, whose `addWallet` is a pure no-op. Since `ComplianceServicePermissionless` never consults investor-registry data for authorization in the first place (only the denylist/blacklist checks gate transfers), the fix as suggested would be dead code on the one compliance path where it could matter.

On special wallets (`WalletManager::setSpecialWallet`, issuer/platform/exchange) assigning any of these roles is already gated by `onlyIssuerOrAbove` a privileged, internal action taken by the issuer/platform operator, not a customer-facing self-service flow like investor onboarding. Platform/issuer/exchange wallets are Securitize/issuer-controlled operational infrastructure, not third-party accounts, so the entity granting the role already knows who it's assigning it to.

7.2 Informational

7.2.1 Deploy task implicit `GlobalDenyListManager::initialize` resolution can silently deploy an uninitialized proxy

Description: The `deploy-denylist-manager` task calls `hre.upgrades.deployProxy(GlobalDenyListManager, [])` with no explicit initializer option. In `@openzeppelin/hardhat-upgrades`, `getInitializerData` sets `allowNoInitialization = true` when the initializer option is omitted and the args array is empty - exactly this call. It then resolves a function named `initialize`; on a failed lookup it returns `0x` instead of throwing.

The lookup currently succeeds, so `GlobalDenyListManager::initialize` runs by `delegatecall` from the `ERC1967Proxy` constructor and deployment plus initialization are atomic. The deployer receives `DEFAULT_ADMIN_ROLE` in the same transaction that creates the proxy, leaving no front-running window. The finding is the fragility of that guarantee, not a live defect.

Impact: If `initialize` is renamed or replaced by a differently named initializer, the task deploys the proxy with empty init data and reports success. `initialize` is unauthenticated and grants `DEFAULT_ADMIN_ROLE` to `msg.sender`, and that role gates `BaseRBACContract::_authorizeUpgrade`, so the first caller of the uninitialized proxy gains full upgrade control. The test suite would not catch the regression: the upgrade test passes initializer: `'initialize'` explicitly while the production task does not.

Recommended Mitigation: Name the initializer explicitly so a rename throws, and assert the post-deploy state:

```
const denylistManager = await hre.upgrades.deployProxy(GlobalDenyListManager, [], {
  initializer: 'initialize',
});
await denylistManager.waitForDeployment();

const [deployer] = await hre.ethers.getSigners();
if ((await denylistManager.getInitializedVersion()) !== 1n) throw new Error('proxy not initialized');
if (!(await denylistManager.isAdmin(deployer.address))) throw new Error('deployer is not admin');
```

The explicit initializer option turns the silent skip into a thrown error, while `BaseRBACContract::getInitializedVersion` and `GlobalDenyListManager::isAdmin` assert the on-chain end state independently of plugin behavior.

Securitize: Fixed in commit [3e0e13a](#).

Cyfrin: Verified.

7.2.2 `deploy-all` does not await `GlobalDenyListManager::addOperator`, `changeAdmin` receipts and reports success on unconfirmed transactions

Description: `deploy-all` sends the two role handover transactions as `await globalDenylistManager.addOperator(...)` and `await globalDenylistManager.changeAdmin(...)`. In ethers v6 that promise resolves to a `ContractTransactionResponse` once the node accepts the transaction into the mempool, not once it is mined; the resolved object exposes `wait` and carries no status. Neither call invokes `wait`, so the task returns and the process exits with both transactions still unconfirmed.

Impact: The task prints its handover lines and exits zero regardless of the on-chain outcome; the role transfer transactions may never end up successfully executing but the deployer mistakenly believes they have because of the print output.

This outcome conflicts with the following section in `GlobalDenylist-AuditScope.md` which states:

S-1 - Admin handover silently fails, leaving the deployer as permanent root admin

`deploy-all` calls `changeAdmin` and returns without asserting the outcome. If the target already held `DEFAULT_ADMIN_ROLE` (granted out of band through the permissive `grantRole` override), Operations believes the ephemeral deployer key was decommissioned while it retains upgrade, pause, and role-granting authority over a platform-wide contract. A short-circuit variant of this was found and fixed during development; verify no residual path reproduces it.

Recommended Mitigation: Await the receipt for each role transaction, keeping both calls inside their existing zero-address guards:

```
if (args.operator !== ZeroAddress) {
  await (await globalDenylistManager.addOperator(args.operator)).wait();
}
if (args.admin !== ZeroAddress) {
  await (await globalDenylistManager.changeAdmin(args.admin)).wait();
}
```

`wait` resolves only after the transaction is mined and throws `CALL_EXCEPTION` when the receipt carries a status of zero, which is exactly the revert case pre-flight gas estimation cannot catch. It also throws when the transaction was replaced at the same nonce. Bound the call as `wait(1, timeoutMs)` so a stuck transaction cannot block indefinitely, since the default timeout is zero, and raise the confirmation count above the default of one on chains where reorg depth matters.

Securitize: Fixed in commit [4f7632d](#).

Cyfrin: Verified.

7.2.3 Denylist mutators always return true, discarding the `EnumerableSet` result

Description: `addToGlobalDenylist` and `removeFromGlobalDenylist` end in a literal `return true`:

```
function addToGlobalDenylist(address wallet) external override onlyRole(OPERATOR_ROLE) whenNotPaused
→ addressNotZero(wallet) returns (bool) {
  _addToGlobalDenylist(wallet);
  return true;
}
```

`EnumerableSet.add` returns true only if the wallet was not already present, and `.remove` only if it was present.

The interface asks for the opposite of a constant: [IGlobalDenylistManager.sol:113](#) and [:121](#) declare `@return True` on success. A no-op — adding an already-denylisted wallet, removing one that was never listed — emits no event and returns true, indistinguishable from a call that changed the set. Since there is no enumeration getter, the return is the caller's only per-call signal, and it carries no information.

Recommended Mitigation: Return the helper's result. Event behaviour is unchanged:

```
function addToGlobalDenylist(address wallet) external override onlyRole(OPERATOR_ROLE) whenNotPaused
→ addressNotZero(wallet) returns (bool) {
  return _addToGlobalDenylist(wallet);
}

function _addToGlobalDenylist(address wallet) private returns (bool added) {
  added = _globallyDenylistedWallets.add(wallet);
  if (added) {
    emit WalletAddedToGlobalDenylist(wallet, _msgSender());
  }
}
```

Same for `removeFromGlobalDenylist`. Update the `NatSpec` to `@return True` if the wallet was newly added, false if it was already denylisted (and the mirror for removal). For the bulk functions, either return the count of wallets actually changed or document that the `bool` is always true.

Securitize: Fixed in commit [98c9a3a](#).

Cyfrin: Verified.

7.2.4 GlobalDenyListManager::revokeOperator is bypassable via inherited AccessControlUpgradeable::revokeRole, renounceRole, which emit no OperatorRevoked

Description: GlobalDenyListManager::grantRole is overridden so that granting OPERATOR_ROLE through the generic AccessControl path still emits the domain event OperatorAdded. The two matching removal paths received no such treatment: revokeRole and renounceRole are inherited from AccessControlUpgradeable unchanged and emit only the standard RoleRevoked.

An admin calling revokeRole(OPERATOR_ROLE, operator) therefore removes the role without emitting OperatorRevoked, and an operator calling renounceRole(OPERATOR_ROLE, self) does the same unilaterally. Both bypass GlobalDenyListManager::revokeOperator, the sanctioned removal path and the only one that emits the domain event.

Impact: No OperatorRevoked event is emitted; the asymmetry with the grantRole override is what marks this as an oversight rather than a design choice: the grant side was hardened specifically to keep the domain events authoritative, and the removal side was left inherited.

Recommended Mitigation: Reject OPERATOR_ROLE in both inherited entry points, so every operator removal goes through revokeOperator:

```
function revokeRole(bytes32 role, address account) public virtual override onlyRole(DEFAULT_ADMIN_ROLE)
→ {
    if (role == OPERATOR_ROLE) revert UseRevokeOperator();
    super.revokeRole(role, account);
}

function renounceRole(bytes32 role, address callerConfirmation) public virtual override {
    if (role == OPERATOR_ROLE) revert UseRevokeOperator();
    super.renounceRole(role, callerConfirmation);
}
```

Securitize: Fixed in commits [a09ebfa](#), [4f08c18](#).

Cyfrin: Verified.

7.2.5 ServiceConsumer::setDSService accepts an address with no code, halting all transfers and issuance on the wired token

Description: ServiceConsumer::setDSService writes the supplied address into the service registry with no validation at all: no zero check, no service-id whitelist, and no code-size check.

ComplianceServicePermissionless::_isGloballyDenylisted guards only the address(0) case and calls through for every other value. That call returns bool, so Solidity 0.8 emits an extcodesize check on the target before decoding the return data. A wired address holding no code fails that check and the call reverts with empty revert data.

The deployment task does not compensate. deploy-all resolves the supplied address with ethers.getContractAt, which validates address format and EIP-55 checksum client-side but never queries the chain for code, so an all-lowercase typo or a pasted EOA passes straight through to set-services.

Impact: Wiring a codeless address into the GLOBAL_DENYLIST_MANAGER slot halts the token. transfer, transferFrom, every issueTokens variant, preTransferCheck and getComplianceTransferableTokens all revert. Transfers and issuance stop rather than degrade.

Recommended Mitigation: * Validate in the registry:

```
function setDSService(uint256 _serviceId, address _address) public override onlyMaster returns (bool) {
+   if (_address != address(0) && _address.code.length == 0) revert ServiceAddressHasNoCode();
    services[_serviceId] = _address;
    emit DSServiceSet(_serviceId, _address);
    return true;
}
```

- Validate in the deployment task:

```
if ((await hre.ethers.provider.getCode(args.globalDenylistManagerAddress)) === '0x') {
  throw new Error(`No contract at ${args.globalDenylistManagerAddress}`);
}
// catch interface drift
await globalDenylistManager.isGloballyDenylisted(hre.ethers.ZeroAddress);
```

Securitize: Fixed in commit [cbfcd9e](#).

Cyfrin: Verified; the script guard was added but `ServiceConsumer::setDSService` remains unchanged so technically still possible in other cases.

7.2.6 `deploy-all` treats a zero `--global-denylist-manager-address` as wired, logging success while leaving global denylist enforcement off

Description: `deploy-all` declares `--global-denylist-manager-address` as an optional string parameter defaulting to undefined, and guards the wiring branch with a bare truthiness test. The zero address arrives as the non-empty 42-character string `0x0000000000000000000000000000000000000000000000000000000000000000`, which is truthy in JavaScript, so passing it selects the wired branch.

That branch logs that an existing manager is being used, then resolves the address with `ethers.getContractAt`, which never queries the chain for code. `set-services` subsequently takes both of its own guarded branches, logging one connection line for the token and another for the compliance service while writing `address(0)` into both `GLOBAL_DENYLIST_MANAGER` slots.

The resulting on-chain state is an unset slot, which is precisely the fail-open branch in `ComplianceServicePermissionless::_isGloballyDenylisted`. The token ends up with no global denylist enforcement while three separate log lines state that the manager was wired.

Impact: Omitting the flag and passing the zero address produce identical on-chain state but opposite operator-visible evidence. Omission is silent: the guard is skipped and the deploy log never mentions the global denylist at all. The zero address is actively misleading: the log asserts success three times.

Recommended Mitigation: Reject the zero address explicitly, and make the omitted case visible:

```
if (args.globalDenylistManagerAddress) {
  if (args.globalDenylistManagerAddress === ethers.ZeroAddress) {
    throw new Error('--global-denylist-manager-address cannot be the zero address; omit the flag to
    ↳ deploy unwired');
  }
  console.log(`Using existing shared Global Denylist Manager at: ${args.globalDenylistManagerAddress}`);
  globalDenylistManager = await ethers.getContractAt('IDSGlobalDenylistManager',
  ↳ args.globalDenylistManagerAddress);
} else {
  console.log('WARNING: GLOBAL_DENYLIST_MANAGER left unset - global denylist enforcement is OFF for this
  ↳ token');
}
```

The `else` branch carries as much value as the guard itself: it turns the silent omission into a visible one, so neither path can leave an operator believing enforcement is active when it is not. Aligning both repositories on a single convention for an unset address parameter would remove the underlying inconsistency rather than only its symptom here.

Securitize: Fixed in commit [cbfcd9e](#).

Cyfrin: Verified.

7.2.7 `ComplianceService::validateSeize` screens the seize destination only for special-wallet status, so a globally denylisted special wallet can still receive seized tokens

Description: `ComplianceService::validateSeize` gates the seize destination on a single requirement:

```
function validateSeize(  
    address _from,  
    address _to,  
    uint256 _value  
) public virtual override onlyToken returns (bool) {  
    require(getWalletManager().isIssuerSpecialWallet(_to), "Target wallet type error");  
  
    return recordSeize(_from, _to, _value);  
}
```

Notably `_to` is never checked against the global denylist, only that it is a special wallet. While very narrow it is theoretically possible that:

- a special wallet is sanctioned by OFAC
- seize is used to move funds to that special wallet, in breach of OFAC sanctions

Recommended Mitigation: Enforce that the `_to` address where seized funds are sent is not on the global denylist.

Securitize: Acknowledged; platform wallets are managed by the Operations team and are not expected to be affected by this scenario.

7.3 Gas Optimization

7.3.1 Cache identical storage reads in `ComplianceServicePermissionless::_isGloballyDenylisted, _isLocallyBlacklisted`

Description: `ComplianceServicePermissionless::_isGloballyDenylisted` reads `services[GLOBAL_DENYLIST_MANAGER]` twice per call: once directly through `ServiceConsumer::getDSService` for the `address(0)` guard, then again inside `ServiceConsumer::getGlobalDenyListManager`, which is itself a `getDSService` wrapper. `ComplianceServicePermissionless::_isLocallyBlacklisted` has the identical shape against `services[BLACKLIST_MANAGER]`. Each redundant read costs a warm SLOAD, a recomputed keccak256 mapping-slot derivation, and an internal dispatch.

`ComplianceServicePermissionless::checkTransfer` multiplies the waste: it calls each helper once for `_from` and once for `_to`, so a passing transfer performs eight reads of only two distinct storage slots. `checkTransfer` is on the state-changing path of every transfer via `ComplianceService::validateTransfer`, so the cost is borne by every token holder. `ComplianceServicePermissionless::preIssuanceCheck` and `ComplianceServicePermissionless::getComplianceTransferableTokens` call the same helpers and inherit the per-call redundancy.

The address cannot go stale between reads. `services` is written only by `ServiceConsumer::setDSService`, which is onlyMaster, and the interposed `IDSGlobalDenyListManager::isGloballyDenylisted` and `IDSBlackListManager::isBlacklisted` calls are declared view, so the compiler emits `STATICCALL` and no reentrant write is reachable.

Impact: Measured with a `solc 0.8.22` harness (optimizer on, runs 200) reproducing the exact call shape against view manager mocks, on the passing path where neither wallet is listed:

- caching inside both helpers alone saves 507 gas per transfer
- caching plus a two-wallet form for `checkTransfer` saves 886 gas per transfer

Recommended Mitigation: Cache the resolved manager address in a local and reuse it. First fix both helpers in place, so every caller benefits:

```
function _isGloballyDenylisted(address _wallet) internal view returns (bool) {
    address manager = getDSService(GLOBAL_DENYLIST_MANAGER);
    if (manager == address(0)) return false;
    return IDSGlobalDenyListManager(manager).isGloballyDenylisted(_wallet);
}

function _isLocallyBlacklisted(address _wallet) internal view returns (bool) {
    address manager = getDSService(BLACKLIST_MANAGER);
    if (manager == address(0)) return false;
    return IDSBlackListManager(manager).isBlacklisted(_wallet);
}
```

Then add two-wallet variants so `checkTransfer` resolves each manager once for both wallets:

```
function _anyGloballyDenylisted(address _a, address _b) internal view returns (bool) {
    address manager = getDSService(GLOBAL_DENYLIST_MANAGER);
    if (manager == address(0)) return false;
    IDSGlobalDenyListManager denyList = IDSGlobalDenyListManager(manager);
    return denyList.isGloballyDenylisted(_a) || denyList.isGloballyDenylisted(_b);
}

function _anyLocallyBlacklisted(address _a, address _b) internal view returns (bool) {
    address manager = getDSService(BLACKLIST_MANAGER);
    if (manager == address(0)) return false;
    IDSBlackListManager blackList = IDSBlackListManager(manager);
    return blackList.isBlacklisted(_a) || blackList.isBlacklisted(_b);
}

function checkTransfer(
    address _from,
    address _to,
```

```

    uint256 /*_value*/
) internal view virtual override returns (uint256 code, string memory reason) {
    if (_anyGloballyDenylisted(_from, _to)) {
        return (102, WALLET_GLOBALLY_DENYLISTED);
    }

    if (_anyLocallyBlacklisted(_from, _to)) {
        return (100, WALLET_BLACKLISTED);
    }

    return (0, VALID);
}

```

This requires importing `IDSBlackListManager` and `IDSGlobalDenyListManager` into `ComplianceServicePermissionless`, since `ServiceConsumer` uses named imports and does not re-export them. Short-circuit ordering and the `address(0)` fail-open semantics are unchanged.

Securitize: Fixed in commit [92903ba](#).

Cyfrin: Verified.